

IT7075 · APPLIED AI FOR CYBERSECURITY · MODULE 2

Tools & Environment

Stand up the toolchain you will use all semester — local first, plus an optional free GPU virtual machine in the cloud.

Four parts · two hands-on labs · Lab A required, Lab B optional

Set it up once; reuse it for sixteen weeks

PART 1

Why Tooling Is a Security Topic

Reproducibility, isolation and secrets hygiene are not setup chores — they are the properties that make your work defensible.

MODULE MAP

- ☒ **Why Tooling Is a Security Topic**
- ☐ Set Up Locally
- ☐ Your Machine in the Cloud
- ☐ Your Environment & the Labs

Three reasons your environment is a security control

Each of these is a property auditors ask about — not a preference.

Reproducibility

"Can you re-run that and get the same answer?"

If an analysis cannot be reproduced, it cannot be defended in an incident report, a court filing, or a code review. Pinned versions and a recorded environment are what make a result a finding rather than an anecdote.

Isolation

"Where exactly did that sample run?"

This course handles suspicious code and runs attacks. That work stays inside containers and VMs on a network with no route to anything real. Isolation is what separates a lab exercise from an incident of your own making.

Secrets hygiene

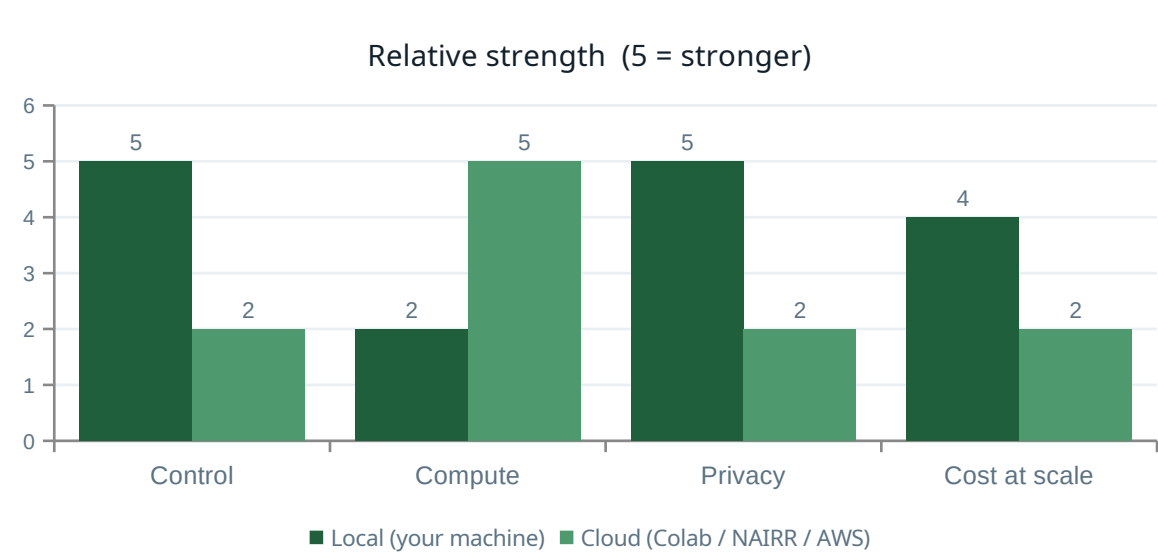
"Is that key still valid?"

API keys and credentials are live liabilities from the moment they exist. Where they are stored, what excludes them from Git, and how fast you can rotate them are operational questions with immediate consequences.

Property	The habit that delivers it	What it costs you to skip
Reproducibility	A venv per project; pinned versions; the notebook committed	A result you cannot defend when someone asks how you got it
Isolation	Docker and VMs on a host-only network; snapshots before you break things	Your own machine becomes the incident
Secrets hygiene	.env excluded by .gitignore; Colab Secrets; a rotation habit	A key on GitHub, found by a bot in minutes, billed to you

Local and cloud — pick by data, not by preference

Neither is better. The question is what your data is and how much compute you need.



If your data is...	Run it...
Real logs, PII, client material	Locally, or on a VM you control
Synthetic or provided samples	Anywhere — Colab is fine
Too big for your RAM	Cloud, after you have de-identified it
Needing a GPU you don't own	Colab for a session; NAIRR for the term

The line that actually matters: "Cloud" means your data is processed on someone else's hardware under their terms. That is a decision, and it should be a deliberate one.

This week's order: local first, always. The cloud GPU is a supplement you add in Lab B, not a substitute for having a working machine.

Three homes, one course

Your laptop, a borrowed GPU, and a GPU you keep. The axis that actually separates them is persistence.

“Colab is a GPU you borrow for a session. The NAIRR VM is a GPU you keep for the term.”

YOUR LAPTOP	COLAB	NAIRR VM
yours, always	a GPU you borrow	a GPU you keep
no GPU worth using	T4 · 16 GB, free tier	A100 slice · 20 GB
your own data, local	resets when the session ends	persists all term
nothing to provision	nothing to provision	~10 min to provision
the editor lives here	good for one sitting	a Linux box you own

The axis that matters is persistence — everything else follows from it.

The VM is optional — say so early. Everything graded in this module can be done with a laptop and Colab. The NAIRR lab is 25 marks of optional extra credit, and it exists mainly so module 3 has somewhere persistent to live.

The toolkit: six categories

The whole semester's tooling on one slide. Tackle it in chunks, in this order.

1 Editor & language

CORE

VS Code · Python 3 · a venv per project

2 Notebooks

CORE

Jupyter locally · Google Colab + Drive

3 Version control

CORE

Git · GitHub · your course repository

4 The security lab

SECURITY

VirtualBox/VMware · Docker · Kali + targets

5 AI model access

AI

Cloud APIs · Ollama / LM Studio · Hugging Face

6 Files & secrets

CRITICAL

Drive or local storage · .env · Colab Secrets

Order of attack: one through three are your day-to-day workspace and everything else assumes them — get those working before anything else. Four is the security lab and can wait until you need it. Five is how your code reaches a model. Six is the one that is easy to defer and expensive to get wrong.

Six categories, one working environment

The lab does not grade depth in any one of these. It grades whether all six are connected and talking to each other.

Graded on a working, connected environment — breadth, not mastery of any one tool

Editor VS Code + the Python extension	Language Python 3 in a venv, not the system one	Notebooks JupyterLab locally, Colab in the cloud
Version control Git and GitHub — your own repo	Secrets .env, git-ignored; Colab Secrets	Model access an API key, Colab, or your own VM

GitHub + own repo	Colab + Drive	VS Code + Python + Jupyter	Secure key handling	Forum reflection
15	25	30	20	10

PART 2

Set Up Locally

The required Lab A path — editor, notebooks, version control, secrets, model access, and the isolated security lab.

MODULE MAP

- ☐ Why Tooling Is a Security Topic
- ☒ **Set Up Locally**
- ☐ Your Machine in the Cloud
- ☐ Your Environment & the Labs

Editor and language: VS Code, Python, venv

One editor for code and notebooks, and one isolated set of packages per project.

Create and use a virtual environment

```
python3 -m venv .venv          # one per project, in the project folder
source .venv/bin/activate      # Windows: .venv\Scripts\activate

pip install openai python-dotenv pandas jupyter
pip freeze > requirements.txt  # so someone else can reproduce this

# In VS Code: Ctrl/Cmd+Shift+P -> "Python: Select Interpreter"
# -> choose the one inside .venv
```

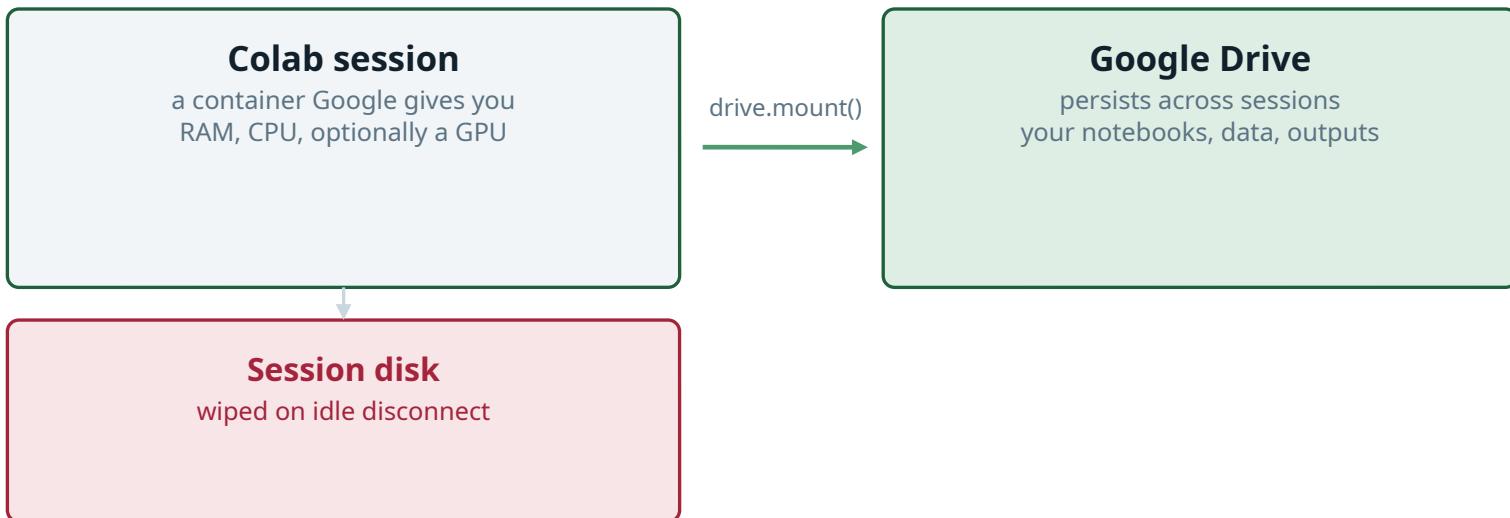
Use	For
Notebooks (.ipynb)	Exploring, showing your work cell by cell, lab submissions
Scripts (.py)	Anything you run repeatedly, or ship as an application

Install	Why
VS Code	One editor for scripts and notebooks
Python 3	The language everything here is written in
Python extension	Interpreter selection, linting, debugging
Jupyter extension	Run .ipynb notebooks inside the editor

The error you will hit: "ModuleNotFoundError" for a package you just installed almost always means VS Code is using the system interpreter, not your venv. Check the interpreter shown in the status bar first.

Notebooks in the cloud: Colab and Drive

Free, hosted, a GPU one menu away — and everything on the session disk disappears.



Lives where	Survives?
/content/	No — wiped
pip installs	No — reinstall
/content/drive/	Yes
The notebook itself	Yes, if saved

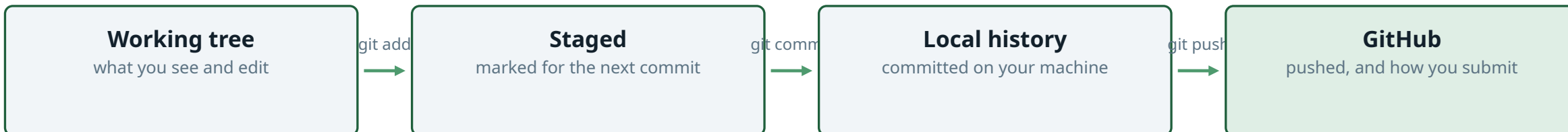
Course convention: mount Drive and keep everything under one folder — Drive/AppliedAI-Sec/ with a subfolder per module. Then a session reset costs you nothing but the reinstall, and your work is in one place when the capstone asks for it.

One line, authorise once

```
from google.colab import drive
drive.mount('/content/drive')
# Drive now appears as a folder you can read and write
```

Version control: Git and GitHub

Four states, three commands. That is all you need for this course.



The whole workflow, once per work session

```

git status          # what changed?
git add module2/lab_a.ipynb  # stage just what you mean to commit
git commit -m "Lab A: environment verified locally and in Colab"
git push            # now it is on GitHub, and submitted
  
```

Habit	Why it is graded
Commit early, commit often	Your history shows how you worked, not just the result
Write real commit messages	"update" tells a reviewer nothing
Never commit .env	See the next slide

Worth an hour of your time: a short Git tutorial on LinkedIn Learning (free to you) or YouTube. For this course you only need add, commit and push — but your repository doubles as a portfolio that employers read, and that is worth doing properly. Your AI assistant in VS Code will happily run these for you once you know what they mean.

Secrets hygiene — the one that is not optional

A key on a public repository is found by an automated scanner in minutes, not days.

The pattern: three files, one of which never leaves your machine

```
# .env                <- real values, NEVER committed
OPENAI_API_KEY=sk-proj-...

# .gitignore          <- committed, and does the protecting
.env
*.key
.venv/

# .env.example        <- committed, so others know what to set
OPENAI_API_KEY=
```

IF A KEY LEAKS

- Rotate it first. Investigate second. The window is the whole risk.
- Deleting the line does not help — Git history keeps it forever.
- Assume compromise from the moment of the push, not from discovery.

Where you are	How the key gets in
Local script	.env + load_dotenv()
Local shell	export in your shell profile
Colab notebook	The Secrets panel (key icon, left sidebar)
A VM	An environment variable in the service unit

Colab specifically: use the Secrets panel, not a variable in a cell. The cell output gets saved into the notebook file, and the notebook gets committed.

Also today: set a hard spend limit in the provider console.

The one script that proves it works

`verify\_env.py` is twelve lines. Read it before you run it — and notice what its guard does not catch.

Tools_and_Environment/code/verify_env.py — COMPLETE

```
import os
from dotenv import load_dotenv, find_dotenv

load_dotenv(find_dotenv()) # walks UP from this file

key = os.getenv("OPENAI_API_KEY")
if not key:
    raise SystemExit("No OPENAI_API_KEY found. Copy sample.env to .env")

from openai import OpenAI
client = OpenAI(api_key=key)
resp = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user",
                "content": "Say hello in one short sentence."}]
)
print("Environment OK — model replied:")
print(resp.choices[0].message.content)
```

The guard the placeholder walks straight through

`if not key:` is true only for absent or empty. A freshly copied `sample.env` still holds the literal `sk-...` — non-empty, so the guard passes and the student gets a traceback instead of the message written for them.

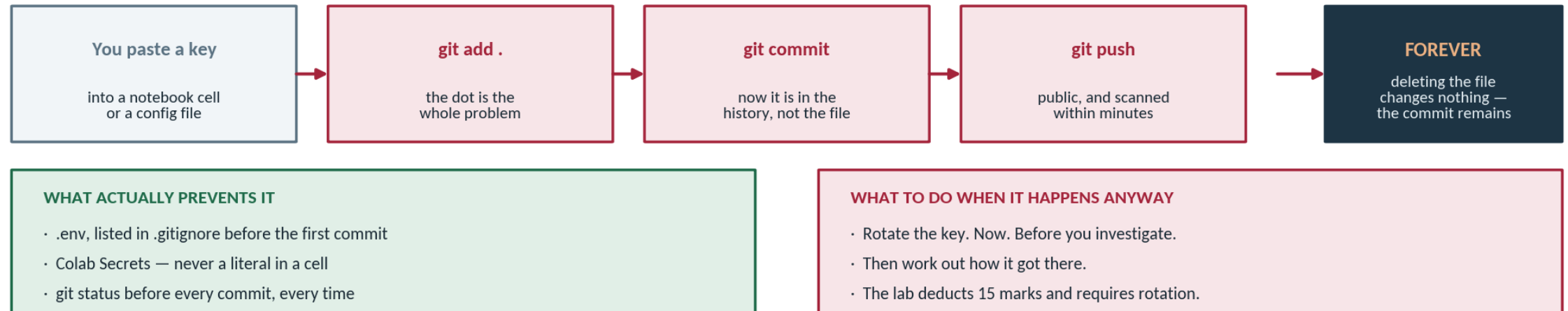
VALIDATE THE SHAPE, NOT JUST THE PRESENCE

```
if not key or key.startswith("sk-..."):
    raise SystemExit("Edit .env — that is the placeholder")
```

Two more things that will cost somebody an hour. There is no try/except around the API call, so a 429 quota error or a blocked network also produces a raw traceback — and the solutions key anticipates exactly that error. And `sample.env` ships an `ANTHROPIC\_API\_KEY` that this script never reads, so a student holding only an Anthropic key is told their setup is broken when it is not.

How a key actually escapes

Not by being stolen. By being committed — and the commit is the part you cannot take back.



Automatic –15 and mandatory rotation if a live key appears anywhere in your submission.

AI model access: three paths to a model

Get at least one working this week. You compare them properly in Module 3.

Cloud API

OpenAI · Anthropic

Easiest and most capable. Messages in, text out. You pay per token and your prompt leaves your machine.

Local LLM

Ollama · LM Studio

Open-weight models on your own hardware. Private and offline. Limited by your RAM and whether you have a GPU.

Hugging Face

The hub

Where open models and datasets live. Not a way to run them so much as where you get them from.

Two paths, nearly the same shape

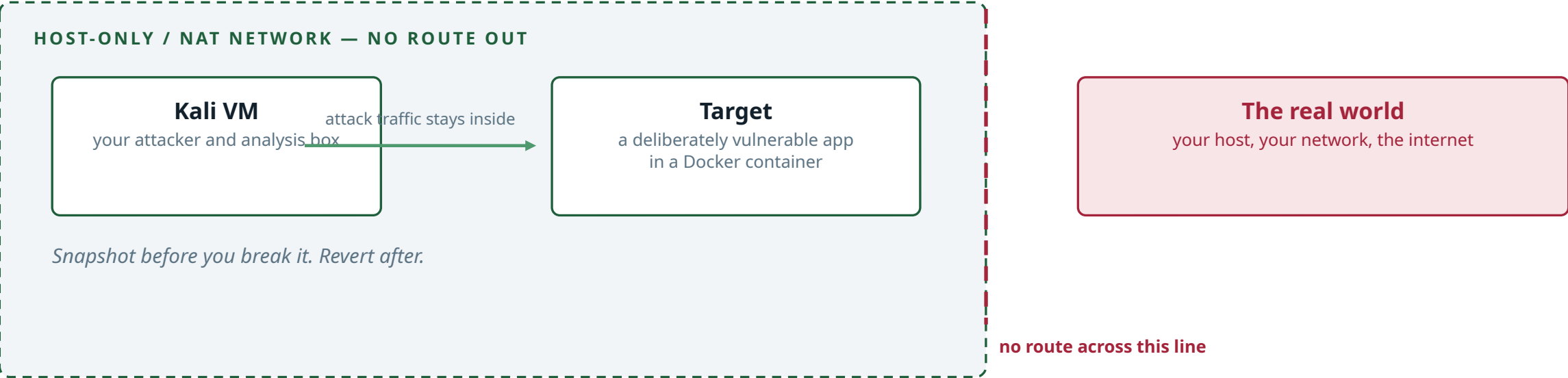
```
# 1 – hosted
from openai import OpenAI
client = OpenAI()
r = client.chat.completions.create(model="gpt-4.1-mini", ...)

# 2 – local, after: ollama pull llama3.1:8b
import ollama
r = ollama.chat(model="llama3.1:8b", messages=[...])
```

	Cloud API	Local LLM
Cost	Per token	Free after hardware
Privacy	Leaves your machine	Stays put
Capability	Highest	Good, if it fits
Works offline	No	Yes

The security lab: VMs, Docker and Kali

One rule governs all of it — the lab has no route to anything real.



Tool	What it gives you	The rule attached to it
Docker	A tool or vulnerable app packaged with its dependencies, disposable	Containers share the host kernel — isolation, not a security boundary against a determined escape
A VM (VirtualBox/VMware)	A full guest OS you can snapshot and revert	Set the adapter to host-only or NAT before you boot it, not after
Kali Linux	The attacker and analysis box, tools preinstalled	Only ever point it at lab targets you own

The Python you will read

Same message as Module 1 — reading is the skill, and the bar is lower than you think.

One pandas line seeds a brute-force detector

```
import pandas as pd

logs = pd.read_csv("auth.log.csv")
fails = logs[logs["result"] == "FAIL"]

noisy = fails.groupby("src_ip").size()
print(noisy[noisy > 10].sort_values(ascending=False))
```

Why this matters here specifically: in this course an AI assistant will often write the first draft of your code. Reading is what lets you tell whether that draft does what you asked, and it is the difference between using a tool and being used by one.

You need	Not
To follow this	To have written it
To fix an error in it	To design the library
To adapt it to your data	To optimise it
To spot it doing something odd	To review it formally

If Python is new

The recommended Weeks 1–3 review from Module 1 still stands, and it is still not graded:

- LinkedIn Learning — "Python for Non-Programmers" (free to you)
- learnpython.org — interactive, no install
- A free two-hour YouTube course

PART 3

Your Machine in the Cloud

Optional, and the companion to Lab B: a free GPU virtual machine on NAIRR Jetstream2 for the day a job outgrows your laptop.

MODULE MAP

- ☐ Why Tooling Is a Security Topic
- ☐ Set Up Locally
- ☒ **Your Machine in the Cloud**
- ☐ Your Environment & the Labs

Why a cloud VM — and what NAIRR gives you

Optional this term. It becomes the compute home for the later modules.

A100

GPU slice on a g3.large instance

~20 GB

of VRAM — fits a 14B model comfortably

\$0

to you, through the course allocation

All term

it persists — unlike a Colab session

When your laptop is enough	When you want Colab	When you want the VM
Reading notebooks; API calls; small pandas work	A GPU for one session, and you do not mind reinstalling	A model that must still be loaded tomorrow
Anything CPU-bound and small	A quick experiment you will not repeat	Data you do not want on a shared consumer service
Most of Modules 1 and 2	Most one-off GPU experiments	Running your own model server; the later builds

What NAIRR actually is: the National AI Research Resource — a US programme granting academic access to real cloud infrastructure. Jetstream2 is the system we use. It behaves like AWS: you provision an instance from a web console and reach it over SSH. The skill transfers directly.

What the VM actually is

Not “a cloud GPU” in the abstract — this is the reference instance, measured.

WHAT ``nvidia-smi``, ``nproc`` AND ``free -h`` ACTUALLY REPORT ON THE REFERENCE `g3.large`

GPU	VRAM	CUDA	CPU	RAM	DISK
GRID A100X-20C	20 480 MiB	12.2	16 cores	≈ 60 GiB	100 GB
a partial A100 — not a whole card	2 157 MiB used at idle ≈ 20 GB to spend	driver 535.309.01	<code>`nproc`</code>	58 GiB available, no swap	the 20 GB default is not enough — models alone are 15 GB+

Measured on the reference run in `Lab_CloudVM_Solutions.md`. Accept any 535.x driver minor.

THE THREE COMMANDS THE LAB ASKS YOU TO RUN

```
$ nvidia-smi
  GRID A100X-20C   | 2157MiB / 20480MiB   | 0%   | P0
  Driver 535.309.01      CUDA Version: 12.2
$ nproc
16
$ free -h
Mem:  59Gi total  1.1Gi used  58Gi available  Swap: 0B
```

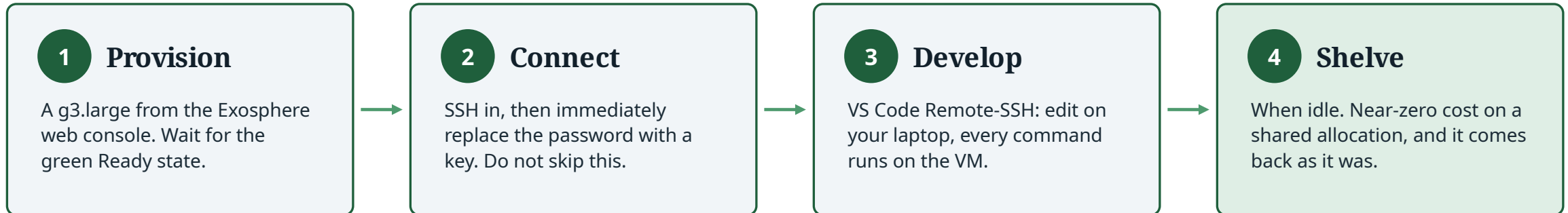
Choose the 100 GB disk at provisioning time

The default is 20 GB and it looks like plenty. Model files in module 3 are 15 GB and up, so you find out two weeks later, mid-download, with no easy way to resize.

Ten minutes to go green — do not declare SSH broken before then.

Provision → connect → develop → shelve

The four-step lifecycle, mirrored exactly by Lab B.



```
~/.ssh/config — then it is just: ssh it7075
```

```
Host it7075
  HostName 149.165.xxx.xxx
  User exouser
  IdentityFile ~/.ssh/id_ed25519
  ServerAliveInterval 60      # stops idle drops mid-session
```

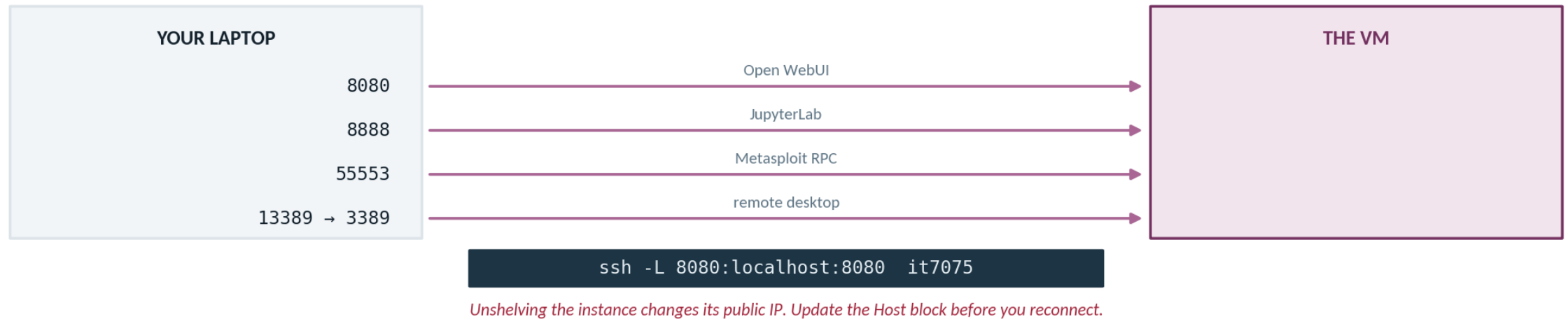
The mental model: Colab is a GPU you rent for a session. The NAIRR VM is a GPU you keep for the term.

Shelve, do not just close the tab. A running idle instance consumes the shared allocation everyone in the class draws from.

Nothing is published — you tunnel in

One SSH connection carries every service you need. No open ports, no public endpoints.

Nothing is published to the internet — the tunnel is the only way in



code/ssh_config_sample — THEN JUST: `ssh it7075`

```
Host it7075
  HostName <INSTANCE-IP>
  User exouser
  LocalForward 8080 localhost:8080 # Open WebUI
  LocalForward 8888 localhost:8888 # JupyterLab
```

Keys, not passwords

Five of the lab's 25 marks are for password-free ed25519 login. It is also what makes the tunnel practical — you will open this connection dozens of times.

PART 4

Your Environment & the Hands-On Labs

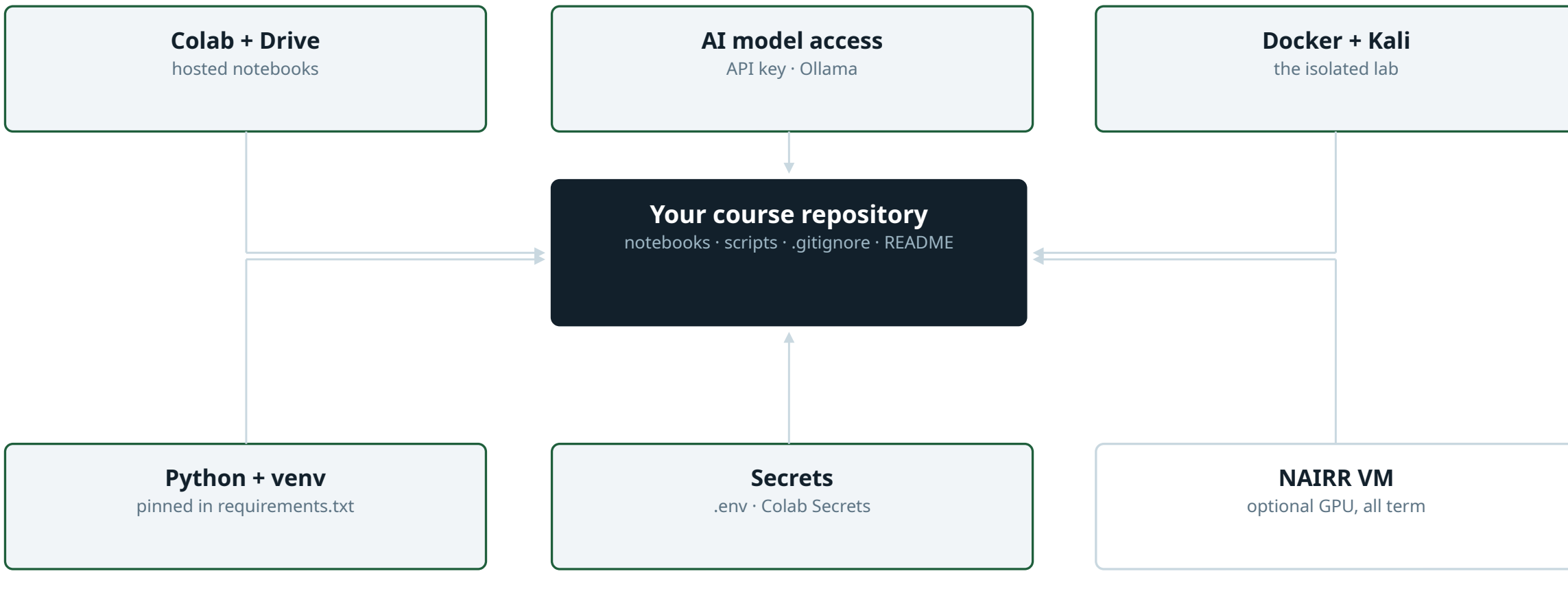
Everything ties together in one repository — and into the two labs that get you set up for the rest of the course.

MODULE MAP

- ☐ Why Tooling Is a Security Topic
- ☐ Set Up Locally
- ☐ Your Machine in the Cloud
- ☒ **Your Environment & the Labs**

Your full course environment

One repository at the centre. Every later module plugs into this same skeleton.



Hands-on: the two labs

Lab A is required and comes first. Lab B whenever you want cloud compute.

REQUIRED

Lab A — Set Up & Verify Your Environment

- Create your GitHub course repository
- Mount Drive in Colab and save a notebook there
- Create a venv locally and install the requirements
- Re-run the Module 1 triage code — in Colab and locally
- Store your key in .env (local) and Colab Secrets (cloud)

OPTIONAL SUPPLEMENT

Lab B — Your Machine in the Cloud

- Provision a g3.large instance on Jetstream2
- Connect over SSH, then move to key-based login
- Set up VS Code Remote-SSH against the VM
- Run something on the GPU and confirm it landed there
- Shelf the instance and bring it back

Lab	What you submit	What I am checking
A (required)	Repo link, plus screenshots of the code running in both places (keys hidden)	That it ran on your machine — and that no key is visible anywhere
B (optional)	Screenshots of the instance, the SSH key login, and a shelved state	That you closed the password path and left the allocation tidy

Recap and what's next

Four things to have working before Module 3.

1 A reproducible local setup

VS Code, Python, a venv, and requirements.txt pinned so someone else can rebuild it.

2 A repository you submit from

On GitHub, with .gitignore already excluding .env before anything is committed.

Also true, and easy to forget

Your environment is a security control. Reproducibility, isolation and secrets hygiene are the three properties that make your work defensible rather than merely finished.

3 Keys handled properly

.env locally, Colab Secrets in notebooks, a spend limit set, and a rotation habit.

4 One model path working

A hosted API or a local model. Either. You compare them properly next week.

Next — LLMs on Local & Cloud: run one model three ways — a hosted API, a GPU you rent for a session, and a GPU you keep — and judge them on cost, privacy, persistence, capability and speed.